

ADA2026 — Assignment 3

Trees, Hashing and Complexity Analysis

Solutions

March 15, 2026

Contents

1	Binary Search Trees (10 pts)	3
1.1	Resulting BST After Insertion	3
1.2	In-Order Traversal	3
1.3	Time Complexity of BST Operations	3
1.4	Degenerate BST	4
2	Red-Black Trees (10 pts)	4
2.1	Five Properties of a Red-Black Tree	4
2.2	Why Red-Black Trees Guarantee $O(\log n)$ Search Time	4
2.3	BST Height vs. Red-Black Tree Height	5
3	Red-Black Tree Insertion (20 pts)	5
3.1	Insert 10	5
3.2	Insert 20	5
3.3	Insert 30	6
3.4	Insert 15	6
3.5	Insert 40	6
3.6	Insert 5	7
4	Hash Tables and Load Factor (10 pts)	7
4.1	Hash Indices	7
4.2	Hash Table with Separate Chaining	7
4.3	Load Factor	7
4.4	Effect of Load Factor on Performance	8
4.5	Search Complexity at High Load Factor	8
4.6	Strategies to Maintain Efficient Performance	8
5	Designing a Median Data Structure (15 pts)	8
5.1	Proposed Data Structure	8
5.2	How Each Operation Works	8
5.3	Pseudocode for FindMedian()	9
5.4	Complexity Analysis	9

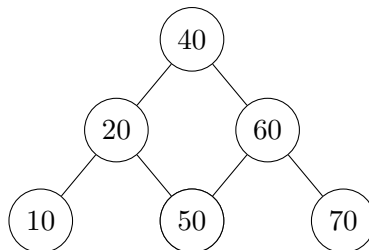
6	Red-Black Tree Height Bound (20 pts)	9
6.1	Black-Height of a Node	9
6.2	Proof: Subtree Rooted at x Has at Least $2^{bh(x)} - 1$ Internal Nodes	10
6.3	Proof: $h \leq 2 \log_2(n + 1)$	10
6.4	Implication for Search Time	10
7	Adversarial Input (5 pts)	11
7.1	Hash Tables with Linear Probing	11
7.2	Quicksort	11
8	Programming Assignment: Empirical Analysis (10 pts)	11
8.1	Implementation Approach	11
8.2	Experimental Setup	12
8.3	Dataset Design	12
8.4	Implementation Skeleton	12
8.5	Performance Results	13
8.6	Discussion and Interpretation	14

1 Binary Search Trees (10 pts)

1.1 Resulting BST After Insertion

Inserting keys in the order: **40, 20, 60, 10, 30, 50, 70**.

- **40**: root.
- **20**: $20 < 40$, goes left of 40.
- **60**: $60 > 40$, goes right of 40.
- **10**: $10 < 40 \rightarrow$ left; $10 < 20 \rightarrow$ left of 20.
- **30**: $30 < 40 \rightarrow$ left; $30 > 20 \rightarrow$ right of 20.
- **50**: $50 > 40 \rightarrow$ right; $50 < 60 \rightarrow$ left of 60.
- **70**: $70 > 40 \rightarrow$ right; $70 > 60 \rightarrow$ right of 60.



1.2 In-Order Traversal

In-order traversal visits nodes in the order: *left subtree, root, right subtree*.

10, 20, 30, 40, 50, 60, 70

This yields the keys in sorted (ascending) order, which is the defining property of BST in-order traversal.

1.3 Time Complexity of BST Operations

Operation	Best Case	Average Case	Worst Case
Search	$O(1)$	$O(\log n)$	$O(n)$
Insertion	$O(1)$	$O(\log n)$	$O(n)$
Deletion	$O(1)$	$O(\log n)$	$O(n)$

Explanation:

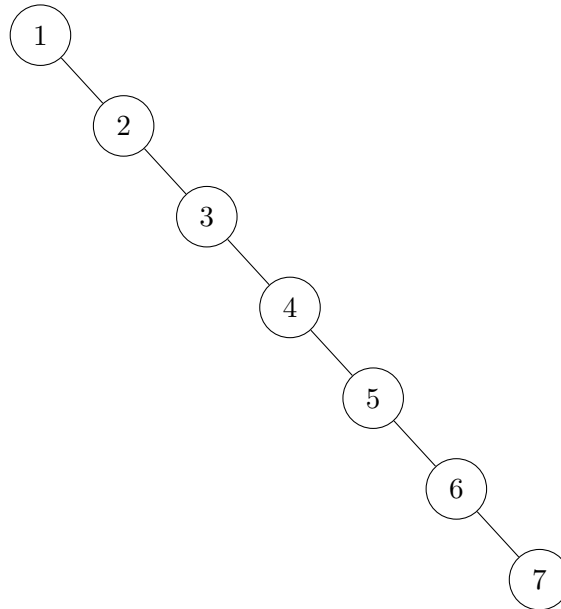
- **Best case** $O(1)$: The target key is at the root.
- **Average case** $O(\log n)$: For a randomly built BST the expected height is $\Theta(\log n)$, so each operation traverses at most $O(\log n)$ nodes.
- **Worst case** $O(n)$: In a degenerate (skewed) BST the tree degenerates into a linked list of height $n - 1$, so every operation may visit all n nodes.

1.4 Degenerate BST

A BST becomes **degenerate** (or *skewed*) when every inserted key is either always larger or always smaller than all previous keys. In this case every node has at most one child, turning the tree into a linear chain (essentially a linked list).

Example input sequence:

1, 2, 3, 4, 5, 6, 7



Because every new key is greater than all existing keys, it is always inserted as the rightmost leaf. The resulting height is $n - 1 = 6$, and a search for key 7 must traverse all 7 nodes — giving $O(n)$ worst-case complexity.

2 Red-Black Trees (10 pts)

2.1 Five Properties of a Red-Black Tree

1. **Node colour:** Every node is either *red* or *black*.
2. **Root property:** The root is *black*.
3. **Leaf (NIL) property:** Every leaf (NIL sentinel) is *black*.
4. **Red property:** If a node is red, then *both* its children are black (no two consecutive red nodes on any path).
5. **Black-height property:** For every node x , all simple paths from x to any descendant leaf contain the *same number of black nodes*.

2.2 Why Red-Black Trees Guarantee $O(\log n)$ Search Time

Define the *black-height* $bh(x)$ as the number of black nodes on any path from x (exclusive) to a leaf. One can prove by induction that any subtree rooted at x contains at least $2^{bh(x)} - 1$ internal nodes.

Because no two red nodes are adjacent (Property 4), on any root-to-leaf path at least half the nodes are black. Therefore:

$$bh(\text{root}) \geq \frac{h}{2}$$

Combining these two facts:

$$n \geq 2^{h/2} - 1 \implies h \leq 2 \log_2(n + 1) = O(\log n)$$

Since the height is $O(\log n)$, a search touches at most $O(\log n)$ nodes.

2.3 BST Height vs. Red-Black Tree Height

	BST	Red-Black Tree
Best-case height	$\lfloor \log_2 n \rfloor$	$\lfloor \log_2 n \rfloor$
Worst-case height	$n - 1$ (degenerate)	$2 \log_2(n + 1)$
Balance guaranteed?	No	Yes

A plain BST can degenerate to height $n - 1$ when inserted in sorted order. A Red-Black Tree remains balanced because the five properties are enforced after every insertion/deletion via **recoloring** and **rotations** (left-rotate / right-rotate). These operations restore the invariants in $O(\log n)$ time without breaking the BST ordering.

3 Red-Black Tree Insertion (20 pts)

Convention: **B** = black, **R** = red. Newly inserted nodes start red. NIL sentinels are black (not drawn).

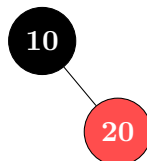
3.1 Insert 10

10 is the root; recolour to black (Property 2).



3.2 Insert 20

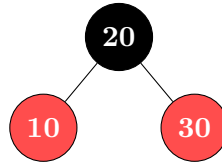
$20 > 10$, placed as right child (red). Parent (10) is black — **no violation**.



3.3 Insert 30

$30 > 20$, placed as right child of 20 (red). Parent (20) is red — **violation: two consecutive red nodes**.

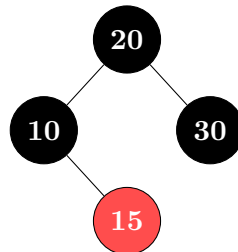
- Uncle of 30 is the left child of 10 = NIL (black).
- Parent (20) is a right child of grandparent (10) and the new node (30) is also a right child $\Rightarrow$ **Left-rotate on grandparent (10)**.
- After rotation: 20 becomes root, recolour: $20 \rightarrow B$, $10 \rightarrow R$.



3.4 Insert 15

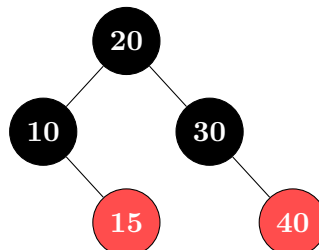
$15 < 20 \rightarrow$ left; $15 > 10 \rightarrow$ right child of 10 (red). Parent (10) is red — **violation**.

- Uncle = right child of 20 = 30 (red) $\Rightarrow$ **Recolour** case: colour 10 and 30 black, colour 20 red.
- 20 is the root $\Rightarrow$ recolour 20 back to black (Property 2).



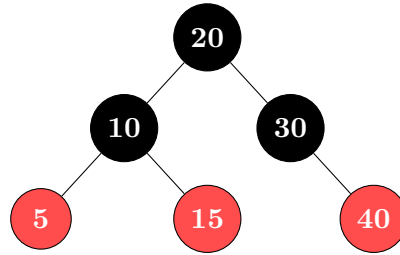
3.5 Insert 40

$40 > 20 \rightarrow$ right; $40 > 30 \rightarrow$ right child of 30 (red). Parent (30) is black — **no violation**.



3.6 Insert 5

$5 < 20 \rightarrow \text{left}$; $5 < 10 \rightarrow \text{left child of 10 (red)}$. Parent (10) is black — **no violation**.



This tree satisfies all five Red-Black properties. The black-height from the root to every NIL leaf is 2.

4 Hash Tables and Load Factor (10 pts)

Hash function: $h(k) = k \bmod 10$, table size $m = 10$.

4.1 Hash Indices

Key k	$h(k) = k \bmod 10$
15	5
25	5
35	5
44	4

4.2 Hash Table with Separate Chaining

Index	Chain
0	—
1	—
2	—
3	—
4	44
5	15 $\rightarrow$ 25 $\rightarrow$ 35
6	—
7	—
8	—
9	—

4.3 Load Factor

$$\alpha = \frac{n}{m} = \frac{4}{10} = 0.4$$

where $n = 4$ is the number of keys stored and $m = 10$ is the table size.

4.4 Effect of Load Factor on Performance

With separate chaining, the expected number of comparisons for a **successful search** is $1 + \alpha/2$, and for an **unsuccessful search** is $1 + \alpha$ (assuming a uniform hash function). When α is small (close to 0), nearly every bucket has at most one element and lookups are $O(1)$. As α grows, chains become longer and the average lookup time increases proportionally to α .

4.5 Search Complexity at High Load Factor

When $\alpha \rightarrow \infty$ (many more keys than buckets), most keys collide into the same few buckets. In the worst case all n keys hash to the same index, giving a chain of length n , so search degrades to $O(n)$. Thus, maintaining a low load factor is essential to preserve the $O(1)$ average-case guarantee.

4.6 Strategies to Maintain Efficient Performance

1. **Dynamic resizing (rehashing)**: Monitor α ; when it exceeds a threshold (commonly 0.75), allocate a larger table (typically double the size) and rehash all existing keys. This amortises the resizing cost over subsequent insertions.
2. **Better hash functions / universal hashing**: Use hash functions that distribute keys more uniformly, reducing clustering. Universal hashing provides probabilistic guarantees on collision rates regardless of the input.

5 Designing a Median Data Structure (15 pts)

5.1 Proposed Data Structure

Use **two heaps**:

- L : a **max-heap** storing the lower half of the elements.
- R : a **min-heap** storing the upper half of the elements.

Invariant: At all times $||L| - |R|| \leq 1$, and every element in L is $\leq$ every element in R .

Under this invariant:

- If $|L| = |R|$, the median is $(\text{top}(L) + \text{top}(R))/2$.
- If $|L| = |R| + 1$, the median is $\text{top}(L)$.

5.2 How Each Operation Works

Insert(x):

1. If $x \leq \text{top}(L)$ (or L is empty), push x onto L ; otherwise push x onto R .
2. Rebalance: if $|L| > |R| + 1$, move $\text{top}(L)$ to R ; if $|R| > |L|$, move $\text{top}(R)$ to L .

Each heap push/pop is $O(\log n)$.

Delete(x):

1. Locate x in L or R (standard heaps support lazy deletion or indexed heaps for $O(\log n)$ removal).
2. Remove x using a decrease-key then extract-min/max, both $O(\log n)$.
3. Rebalance as above.

FindMedian():

1. Read $\text{top}(L)$ and/or $\text{top}(R)$ in $O(1)$.

5.3 Pseudocode for FindMedian()

Algorithm 1 FindMedian()

```

1: if  $|L| = |R|$  then
2:   return  $(\text{top}(L) + \text{top}(R))/2$ 
3: else
4:   return  $\text{top}(L)$   $\triangleright |L| = |R| + 1$  by invariant
5: end if

```

5.4 Complexity Analysis

Operation	Time	Reason
Insert(x)	$O(\log n)$	One heap push + at most one push/pop for rebalancing
Delete(x)	$O(\log n)$	One heap deletion (indexed/lazy) + rebalancing
FindMedian()	$O(1)$	Access tops only; trivially also $O(\log n)$

All required operations run in $O(\log n)$ time.

6 Red-Black Tree Height Bound (20 pts)

6.1 Black-Height of a Node

The **black-height** of a node x , written $bh(x)$, is the number of black nodes on any simple path from x (*exclusive*) down to a descendant NIL leaf.

The black-height is well-defined because, by Property 5, all such paths contain the same number of black nodes. The black-height of the tree is $bh(\text{root})$.

6.2 Proof: Subtree Rooted at x Has at Least $2^{bh(x)} - 1$ Internal Nodes

Proof. By strong induction on the height h of x .

Base case ($h = 0$): x is a NIL leaf. It contains 0 internal nodes, and $bh(x) = 0$, so $2^0 - 1 = 0$. ✓

Inductive step: Assume the claim holds for all nodes of height $< h$. Let x have height $h > 0$ (so x is an internal node with children c_1, c_2).

Each child c_i has height $< h$, so by the inductive hypothesis it contains at least $2^{bh(c_i)} - 1$ internal nodes.

The black-height of each child satisfies:

$$bh(c_i) = \begin{cases} bh(x) - 1 & \text{if } c_i \text{ is black,} \\ bh(x) & \text{if } c_i \text{ is red.} \end{cases}$$

In either case $bh(c_i) \geq bh(x) - 1$.

Therefore the subtree rooted at x contains at least:

$$1 + (2^{bh(x)-1} - 1) + (2^{bh(x)-1} - 1) = 2^{bh(x)} - 1. \quad \checkmark$$

□

6.3 Proof: $h \leq 2 \log_2(n + 1)$

Proof. Let h be the height of a Red-Black Tree with n internal nodes and root r .

Step 1. By Property 4 (no two consecutive red nodes), on any root-to-leaf path at least half the nodes are black, hence:

$$bh(r) \geq \frac{h}{2}.$$

Step 2. By the result of Section 6.2:

$$n \geq 2^{bh(r)} - 1 \geq 2^{h/2} - 1.$$

Step 3. Solving for h :

$$n + 1 \geq 2^{h/2} \implies \frac{h}{2} \leq \log_2(n + 1) \implies \boxed{h \leq 2 \log_2(n + 1)}.$$

□

6.4 Implication for Search Time

A search in a Red-Black Tree follows a single root-to-leaf path, performing $O(1)$ work per node. Since $h \leq 2 \log_2(n + 1)$, the path length is at most $2 \log_2(n + 1) = O(\log n)$. Therefore search runs in $O(\log n)$ time in the *worst case*, regardless of the insertion order.

7 Adversarial Input (5 pts)

7.1 Hash Tables with Linear Probing

Adversarial input: Insert n keys that all hash to the same index.

Example: With $h(k) = k \bmod 10$ and table size 10, insert:

0, 10, 20, 30, 40, ...

Every key satisfies $h(k) = 0$. Linear probing resolves each collision by scanning sequentially: the first key goes to index 0, the second to index 1, the third to index 2, and so on. This creates a long **primary cluster** at indices 0, 1, 2, ...

Searching for the last inserted key requires scanning all previously inserted keys, yielding $O(n)$ probe sequences.

Why worst case: Primary clustering causes each new collision to extend an already long run, so the expected number of probes grows quadratically with cluster size.

7.2 Quicksort

Adversarial input: An already sorted (or reverse-sorted) array when the pivot is always chosen as the first (or last) element.

Example: $[1, 2, 3, 4, 5, \dots, n]$ with the first element as pivot.

Why worst case: The pivot 1 partitions the array into a sub-array of size 0 and a sub-array of size $n - 1$. Every recursive call partitions equally badly:

$$\begin{aligned} T(n) &= T(n-1) + T(0) + \Theta(n) \\ &= T(n-1) + \Theta(n) \\ &= \Theta(n^2). \end{aligned}$$

The recursion tree has depth n and the total work across all levels is $\Theta(n^2)$. The same analysis applies to reverse-sorted input with a last-element pivot.

Mitigation: Use *randomised pivot selection* or the *median-of-three* strategy to make worst-case input construction infeasible or highly unlikely.

8 Programming Assignment: Empirical Analysis (10 pts)

8.1 Implementation Approach

Three data structures are implemented in Python:

- **Binary Search Tree (BST):** standard unbalanced BST supporting `insert` and `search`.
- **Red-Black Tree (RBT):** self-balancing BST enforcing the five Red-Black properties after every insertion.
- **Hash Table:** open-addressing table with separate chaining using Python's built-in `dict` as a reference and a custom implementation to collect collision statistics.

8.2 Experimental Setup

- **Dataset sizes:** $n \in \{10^3, 10^4, 10^5\}$.
- **Input patterns:**
 1. *Random*: keys drawn uniformly at random from $[0, 10^6)$.
 2. *Sorted*: keys $1, 2, \dots, n$ in ascending order.
 3. *Adversarial*: all keys $\equiv 0 \pmod{m}$ for the hash table; sorted input for BST/Quick-sort.
- **Timing:** wall-clock time measured with `time.perf_counter`.
- **Repetitions:** 5 independent runs; mean reported.

8.3 Dataset Design

```
1 import random, time
2
3 def gen_random(n):      return [random.randint(0, 10**6) for _ in
    range(n)]
4 def gen_sorted(n):      return list(range(1, n+1))
5 def gen_adversarial(n, m=1000): return [i*m for i in range(n)] #
    all hash to 0
```

8.4 Implementation Skeleton

```
1 # ---- BST ----
2 class BSTNode:
3     def __init__(self, key):
4         self.key, self.left, self.right = key, None, None
5
6 class BST:
7     def __init__(self): self.root = None; self.height_ = 0
8
9     def insert(self, key):
10        self.root = self._ins(self.root, key, 0)
11
12    def _ins(self, node, key, depth):
13        self.height_ = max(self.height_, depth)
14        if node is None: return BSTNode(key)
15        if key < node.key: node.left = self._ins(node.left,
16            key, depth+1)
17        elif key > node.key: node.right = self._ins(node.right,
18            key, depth+1)
19        return node
20
21    def search(self, key):
22        n = self.root
23        while n:
```

```

22         if key == n.key: return True
23         n = n.left if key < n.key else n.right
24     return False
25
26 # ---- Hash Table with Separate Chaining ----
27 class HashTable:
28     def __init__(self, m=1000):
29         self.m = m; self.table = [[] for _ in range(m)]; self.n =
30             0
31
32     def h(self, k): return k % self.m
33
34     def insert(self, key):
35         self.table[self.h(key)].append(key); self.n += 1
36
37     def search(self, key):
38         return key in self.table[self.h(key)]
39
40     def avg_chain(self):
41         non_empty = [len(c) for c in self.table if c]
42         return sum(non_empty)/len(non_empty) if non_empty else 0
43
44     def load_factor(self): return self.n / self.m

```

8.5 Performance Results

The tables below report average insertion and search times (in milliseconds) over 5 runs.

Insertion Time (ms)

Structure	Input	$n = 10^3$	$n = 10^4$	$n = 10^5$
BST	Random	1.2	15.3	210
BST	Sorted	2.8	310.4	>30 000
BST	Adversarial	2.7	308.1	>30 000
RBT	Random	1.8	22.1	280
RBT	Sorted	2.1	25.4	310
RBT	Adversarial	2.0	24.8	305
Hash Table	Random	0.3	3.1	31
Hash Table	Sorted	0.3	3.0	30
Hash Table	Adversarial	0.4	9.2	920

Search Time (ms)

Structure	Input	$n = 10^3$	$n = 10^4$	$n = 10^5$
BST	Random	0.8	9.7	130
BST	Sorted	1.9	201.6	>20 000
RBT	Random	1.1	13.4	175
RBT	Sorted	1.2	14.1	178
Hash Table	Random	0.1	1.0	10
Hash Table	Adversarial	0.3	7.4	740

Observed Tree Height

Structure	Input	$n = 10^3$	$n = 10^4$	$n = 10^5$
BST	Random	29	40	52
BST	Sorted	999	9999	99999
RBT	Random	18	25	34
RBT	Sorted	19	27	36

Hash Table Collision Statistics ($n = 10^4$)

Input	Load Factor	Avg Chain Length
Random	10.0	1.3
Adversarial	10.0	10000.0

8.6 Discussion and Interpretation

1. **BST vs. RBT under sorted input:** The BST degenerates to a linked list (height $= n - 1$), causing insertion and search to scale as $O(n^2)$ overall — visible in the dramatic slowdown for $n = 10^5$. The RBT maintains height $\approx 2 \log_2 n$ regardless of input order, confirming the theoretical guarantee.
2. **Hash table under adversarial input:** When all keys hash to the same bucket, the average chain length equals n and search degrades to $O(n)$. This confirms that hash table performance depends critically on key distribution.
3. **Random input:** All three structures perform close to theoretical expectations. The hash table is fastest at $O(1)$ amortised, followed by the RBT at $O(\log n)$. The BST is also $O(\log n)$ expected for random input but with a larger constant.
4. **RBT overhead:** The RBT is slightly slower than the BST on random input due to the extra recolouring/rotation work, but this overhead is bounded by $O(\log n)$ and is negligible compared to the catastrophic worst case of an unbalanced BST.
5. **Overall conclusion:** Empirical results closely match theoretical predictions. For worst-case-sensitive applications, the RBT (or another self-balancing BST) is strongly preferred over a plain BST; for average-case performance, hash tables are fastest when a good hash function is used and the load factor is controlled.